HPL 与 HPCG 入门：原理、指标和第一周实验

这份文档供队员在做实验前阅读。实际安装、运行命令见 WSL2 启动步骤，每天的实验安排见 第一周实验手册。目前我们在 Windows + WSL2 上做**单节点 CPU 学习实验**；以后换到老师的服务器，需要重新测量，不能直接沿用 WSL2 的成绩。

一、为什么同时测 HPL 和 HPCG

两项程序都用 GFLOP/s（每秒十亿次浮点运算）报告性能，但计算模式不同。它们回答的是两个不同的问题：机器做**大量规则计算**有多快，以及机器执行**稀疏、访存和通信密集的科学计算**有多快。

项目	HPL	HPCG
主要问题	求解稠密线性方程组 $Ax = b$	求解由三维网格生成的稀疏线性方程组
核心计算	分块 LU 分解、大量矩阵乘法	稀疏矩阵向量乘法（SpMV）、对称 Gauss–Seidel（SymGS）、多重网格（MG）与向量运算
主要压力	浮点计算能力、BLAS 效率、面板通信	内存带宽与延迟、缓存、同步和 MPI 通信
常调参数	全局矩阵大小 `N`、块大小 `NB`、进程网格 `P×Q`、线程和绑定	每进程局部网格 `nx×ny×nz`、MPI 进程数、线程数和绑定
结果文件	当前脚本主要保存到 `stdout.log`	我们使用的 HPCG 3.1 生成 `HPCG-Benchmark_*.txt`

不能用 HPL 的高成绩推断 HPCG 也会高。HPL 的矩阵计算能充分复用数据；HPCG 的稀疏访问通常需要为较少的浮点运算搬运较多数据。两者分数也不能直接相加或当作同一种“利用率”。

二、HPL 是怎样工作的

HPL（High Performance Linpack）构造一个 $N \times N$ 的双精度稠密矩阵，并用带部分主元选择的分块 LU 分解求解 $Ax = b$。矩阵被切成 `NB×NB` 的块，按二维进程网格 `P×Q` 分配给 MPI 进程。分解时，面板处理需要进程间交换数据，更新剩余矩阵则大量调用 BLAS 的矩阵运算（特别是 DGEMM）。因此，BLAS、MPI、数据布局和进程绑定都会影响成绩。

HPL 的主矩阵约占 $8N^2$ 字节；实际进程、通信和工作区还会占内存。选 `N` 时先以**可用物理内存**为准，留出余量，避免换页或 OOM。粗略的计算量约为 $2N^3/3$ 次浮点运算，所以 `N` 稍微增大，运行时间也可能明显增加。小 `N` 适合冒烟测试，但不能代表机器的稳定峰值。

HPL.dat 中最先要懂的字段

- `N`：整个测试的矩阵阶数；所有进程共同求解这一个矩阵。
- `NB`：块的边长。过小可能增加面板和通信开销；过大可能使负载与缓存表现变差。没有所有机器通用的最佳值。
- `P`、`Q`：二维 MPI 进程网格，必须满足 `P×Q = MPI ranks`。例如 4 个进程可比较 `1×4` 与 `2×2`。
- `NDIV` 等后续字段：控制递归面板等内部算法。`HPL.dat` **按位置读取，不能随意增删行**。我们此前遇到的 `Value of NDIV less than 2` 就是输入字段错位或不合法导致的，而不是 HPL 二进制没有编译成功。

HPL 完成后应同时核对程序退出码与残差检查结果。结果行中的 GFLOP/s 是性能值；`PASSED` 表示数值求解满足该次测试的残差阈值。仅有 `exit_code=0` 还不足以证明这次结果正确。

三、HPCG 是怎样工作的

HPCG（High Performance Conjugate Gradients）从三维网格构造稀疏矩阵，使用预条件共轭梯度法（PCG）迭代求解。其预条件器涉及多重网格，常见热点包括 SpMV、SymGS、向量更新、点积归约以及 MPI 进程之间的边界数据交换。这些操作的数据复用和访问模式与 HPL 很不一样。

`hpcg.dat` 中的 `nx ny nz` 是**每个 MPI rank 的局部网格尺寸**，不是整台机器的总尺寸；进程数增加而每 rank 局部尺寸不变时，总问题规模也会增加。因此，比较 1、2、4 ranks 的 GFLOP/s 时，要说明是“每进程工作量大致固定”的初步弱扩展观察，不能直接称为同一问题的加速比。

我们目前构建输出含 `-DHPCG_NO_OPENMP`，说明这份 `xhpcg` 是 **MPI-only 版本**。即使把 `THREADS` 设为大于 1，也不能据此宣称完成了 MPI+OpenMP 对照；要做混合并行实验，需另行构建启用 OpenMP 的版本并记录编译选项。

如何读 HPCG 3.1 报告

程序在运行目录生成 `HPCG-Benchmark_*.txt`。重点看：

1. `Final Summary::HPCG result is VALID ...`：本次结果是否有效。
2. 同一行的 `GFLOP/s rating`：用于记录的最终成绩。
3. `Benchmark Time Summary::Total`：基准内部计时。它与外层脚本测的 `elapsed_wall_s` 包含范围不同，数值不必完全一样。
4. SpMV、MG 等分项时间：用于判断时间主要花在哪里。分项计时可能包含嵌套或重叠口径，不能简单把每项相加当总时间。

我们的第一次 1 rank、`32×32×32`、60 秒测试在报告里得到 `VALID`，最终评级为 **5.99262 GFLOP/s**。这是可用的**短时学习结果**，不能直接当作正式提交成绩。`Raw Total` 与最终评级口径不同，记录时统一使用 `Final Summary` 给出的 rating。

四、怎样做可信的调优实验

先固定源码版本、二进制、MPI、数学库、节点和绑定方式，跑出正确基线。之后每轮只改一个变量，例如 HPL 先改 `N`，固定 `N` 后再改 `NB`；HPCG 先改局部网格，之后再比较 rank 数。每组都保存输入、完整报告、退出码和环境信息，并复跑表现较好的配置。成绩波动大时，先检查后台负载、CPU 频率、内存占用和进程绑定。

我们的 WSL2 单节点实验主要学习：**安装与构建、正确性检查、参数含义、记录和比较方法**。WSL2 里没有真实跨节点网络测试；将来迁移到服务器后，需要重新建立基线，再研究多节点 MPI 通信和 GPU 实现。

建议每次记录如下字段：`run_id`、日期、源码/二进制版本、CPU 与内存、`ranks`、`threads`、HPL 的 `N/NB/P/Q` 或 HPCG 的 `nx/ny/nz/runtime`、退出码、验证结论、最终 GFLOP/s、运行时间和修改说明。模板见 runs.csv。**失败运行也要保留**，因为错误的输入、库混用和资源超限都是调优过程的重要证据。

参考资料

- Netlib HPL 项目页
- HPCG 官方仓库
- HPCG 官方网站